



PLEZY TV

Installation Manual

Android TV / Google TV

Amazon Fire TV (Fire OS and Vega OS)

Samsung Tizen TV

Roku TV and Roku players

Plezy 2.10.0

Generated 2026-07-29

Choose the model family that matches the operating system shown in your device settings. Package formats are not interchangeable.

This repository contains five TV targets:

- **Android TV / Google TV:** the upstream Plezy Flutter application. It already has a Leanback launcher entry, TV banner, D-pad UI, TV detection, Watch Next integration, and native Android playback.
- **Amazon Fire TV (Fire OS):** the upstream Plezy Android application built with its dedicated `AMAZON=1` variant. It includes Fire TV hardware detection, renderer/decoder safeguards, D-pad navigation, and both 32-bit and 64-bit ARM libraries in one APK.
- **Amazon Fire TV (Vega OS):** the new `vega-os/` React Native for Vega WebView application. It bundles the TV Plex/Jellyfin client locally, supports D-pad and Back input, and builds an Amazon Vega `armv7` release package (`.vpkg`).
- **Samsung TV:** the new `samsung-tizen/` Tizen Web application. It provides Plex and Jellyfin sign-in, home shelves, libraries, search, details, series/season browsing, resume, HLS playback through Samsung AVPlay, direct-play fallback, and watch-progress reporting.
- **Roku TV / Roku players:** the new `roku/` SceneGraph/BrightScript application. It provides Plex and Jellyfin sign-in, home browsing, libraries, search, details, series/season browsing, resume-aware HLS playback through Roku's native Video node, direct-stream fallback, and watch-progress reporting.

How to use this guide

- 1 Identify the operating system shown in the TV or player settings. The brand name alone is not enough: Amazon now sells both Fire OS and Vega OS devices, while some television brands sell Roku, Google TV, and proprietary models.
- 2 Use the model-family table below to select the correct section and package type.
- 3 Build or locate that model family's artifact.
- 4 Follow only that section's developer-mode and installation steps.
- 5 Verify the checksum, then disable debugging or developer installation when finished.

An APK cannot be installed on Roku, Samsung Tizen, or Amazon Vega OS. A Samsung WGT cannot be installed on Android, Fire TV, or Roku. Using the wrong package normally produces an unsupported-file or invalid-package error.

Model-family selector

The examples help identify the family; the operating system and minimum version are the deciding factors.

Model family	Typical examples	Minimum runtime	Package	Instructions
Android TV / Google TV	NVIDIA Shield, Chromecast with Google TV, Google TV Streamer, Android/Google TV television sets	Android API 25 / Android 7.1	.apk	Android/Google TV
Amazon Fire TV running Fire OS	Fire TV Stick 4K/4K Max models running Fire OS, Fire TV Cube, Fire TV television sets	Fire OS 6 / Android API 25	universal ARM .apk	Amazon Fire OS
Amazon Fire TV running Vega OS	Fire TV Stick 4K Select, 2026 Fire TV Stick HD, and other models whose About screen says Vega OS	Vega OS 1.x; build against current Vega SDK	armv7 .vpkg	Amazon Vega OS
Samsung Tizen TV	Samsung television whose About screen reports Tizen 6.0+	Tizen TV 6.0	owner-signed .wgt	Samsung Tizen

Model family	Typical examples	Minimum runtime	Package	Instructions
Roku TV / Roku player	Roku TV, Roku Express, Streaming Stick, Streambar, Ultra	Roku OS 10.5+	developer .zip or device-signed .pkg	Roku

The Samsung target is intentionally configured for Tizen 6.0 or newer. Do not lower `required_version` in `samsung-tizen/config.xml` without testing JavaScript, AVPlay, and remote-control behavior on the older model.

Contents

How to use this guide	2
Model-family selector	2
What is ready now	5
Shared build artifacts	5
Model family 1: Android TV / Google TV	6
Compatibility and package choice	6
Build the APK	6
Enable developer mode and install with ADB (recommended)	6
Install without ADB	7
Update, replace, or remove Plezy	7
Model family 2: Amazon Fire TV / Fire TV Stick (Fire OS)	8
Compatibility and package choice	8
Vega OS models require the separate Vega package	8
Build the Fire OS APK	8
Enable Fire OS developer options	8
Connect and install the APK with ADB	9
Update, replace, or remove Plezy	9
Model family 3: Amazon Fire TV / Fire TV Stick (Vega OS)	10
Compatibility and package choice	10
Understand what is already built and what is not	10
Prepare a supported Vega build computer	10
Validate the source on Windows before handoff	11
Build the release VPKG on macOS or Ubuntu	11
Register and enable Developer Mode on the Vega Fire TV	11
Install and launch Plezy	12
First launch and playback test	12
Update, remove, troubleshoot, and publish	13
Model family 4: Samsung Tizen TV	14
Compatibility and package choice	14
Understand Samsung signing	14
Install the current VS Code extensions	14

Enable Developer Mode on the TV	14
Connect VS Code to the TV	15
Create the Samsung certificate profile	15
Validate the Plezy source	15
Build the signed WGT in VS Code	16
Install and launch from VS Code	16
Troubleshoot installation	16
First launch and sign-in	16
Troubleshoot the Samsung interface and playback	17
Samsung model and feature limitations	17
Model family 5: Roku TV / Roku player	18
Compatibility and package choice	18
Validate and build	18
Enable Developer Mode	18
Sideload the ZIP	18
First launch and sign-in	18
Update or remove the developer app	19
Package for publication	19
Verify downloaded or copied artifacts	19
License	19

What is ready now

Target	Installable file already made?	Status and remaining action
Android/Google TV	YES	The publisher-signed ARM64 APK is in <code>dist/android-tv/</code> . Enable developer installation and install it.
Fire TV with Fire OS	YES	The universal ARM APK is in <code>dist/fire-tv/</code> . Enable ADB and install it. The local APK is sideload-signed, not Appstore-signed.
Fire TV with Vega OS	NO	Validated source and automated scripts are in <code>dist/vega-os/</code> and <code>vega-os/</code> . Compile the <code>.vpkg</code> on native macOS/Ubuntu with Amazon's Vega SDK, register the device, then install and test it. Amazon does not support the SDK on Windows/WSL.
Samsung Tizen TV	NO	A universal installable WGT cannot be supplied. Samsung requires the WGT to be signed by a Samsung certificate profile that explicitly includes the target TV's DUID. The complete source ZIP is in <code>dist/samsung-tizen/</code> ; connect that TV, create its certificate profile, then build/sign/install the WGT.
Roku	YES	The developer-installable ZIP is in <code>dist/roku/</code> . Enable Roku Developer Mode and upload it. Store publication still requires Roku packaging/signing.

Shared build artifacts

Build output goes under the ignored `dist/` directory:

```
dist/
android-tv/
plezy-android-tv-arm64-v8a-official-2.10.0.apk
plezy-android-tv-arm64-v8a-release.apk
plezy-android-tv-armeabi-v7a-release.apk
SHA256SUMS.txt
fire-tv/
plezy-fire-tv-universal-2.10.0-release.apk
SHA256SUMS.txt
PROVENANCE.txt
vega-os/
plezy-vega-os-source-2.10.1.zip
plezy-vega-os-2.10.1-release.vpkg # after building on macOS/Ubuntu
SHA256SUMS-source.txt
SHA256SUMS.txt # after the VPKG build
samsung-tizen/
plezy-samsung-tizen-source.zip
plezy-samsung-tizen.wgt # only after signing with your certificate
SHA256SUMS.txt
roku/
plezy-roku-2.10.0-roku.2.zip
SHA256SUMS.txt
PROVENANCE.txt
```

Use the `arm64-v8a` APK for nearly all current Android/Google TV hardware. Use `armeabi-v7a` only for an older 32-bit TV device. Run the Android build with `-Universal` if you need one larger APK containing both ABIs.

This prepared workspace includes `plezy-android-tv-arm64-v8a-official-2.10.0.apk`, extracted unchanged from Plezy's official GitHub 2.10.0 release asset. It is publisher-signed and immediately installable on ARM64 Android/Google TV. Its SHA-256 is recorded in the adjacent checksum file. The local build script below is for rebuilding current source or producing other ABIs.

Model family 1: Android TV / Google TV

Compatibility and package choice

Use this section only when the device settings identify the operating system as Android TV or Google TV and its Android version is API 25/Android 7.1 or newer. This includes external players such as NVIDIA Shield and Chromecast with Google TV as well as television sets that explicitly run Android TV or Google TV.

Use the `arm64-v8a` APK for nearly all current hardware. Use `armeabi-v7a` only when the device is known to run a 32-bit ARM userspace. If unsure, build the larger universal APK.

Install-file status: YES. `dist/android-tv/plezy-android-tv-arm64-v8a-official-2.10.0.apk` is already made, publisher-signed, and ready to sideload on an ARM64 Android/Google TV. Build again only if you need a different ABI, a universal APK, or modified source.

Build the APK

Prerequisites:

- 1 Flutter 3.44.0 or newer.
- 2 Android SDK Platform 36, Build Tools 36.1.0, Platform Tools, NDK 29.0.14206865, and CMake 4.1.2.
- 3 A Java version supported by the included Gradle wrapper (Java 21 is known to work).

From the repository root in PowerShell:

```
.\scripts\build_android_tv.ps1 -AndroidSdk "C:\Android\Sdk"
```

For one universal APK:

```
.\scripts\build_android_tv.ps1 -AndroidSdk "C:\Android\Sdk" -Universal
```

The default is an optimized release build signed with the local Android debug key specifically for private sideloading. It is **not** store-signing. For a publisher build, create `android/key.properties` and use your private release keystore; when that file exists, Plezy's Gradle configuration uses it instead of sideload signing.

Enable developer mode and install with ADB (recommended)

- 1 Put the TV and computer on the same trusted local network.
- 2 On the TV, open **Settings > System (or Device Preferences) > About** and press the **Build** entry seven times.
- 3 Open **Developer options** and enable **USB debugging**, **Network debugging**, or **Wireless debugging**. Names vary by manufacturer.
- 4 Find the TV IP address in its network settings.
- 5 Install Android SDK Platform Tools so `adb` is available.
- 6 Connect and install:

```
adb connect TV_IP_ADDRESS:5555
adb devices
adb install -r ".\dist\android-tv\plezy-android-tv-arm64-v8a-official-2.10.0.apk"
```

Accept the debugging authorization prompt on the TV. If the TV exposes Android 11-style Wireless Debugging, use the pairing address and code shown by the TV first:

```
adb pair TV_IP:PAIRING_PORT
adb connect TV_IP:DEBUG_PORT
adb install -r ".\dist\android-tv\plezy-android-tv-arm64-v8a-official-2.10.0.apk"
```

The app appears as **Plezy** in the TV launcher.

Install without ADB

- 1 Copy the correct APK to a USB drive or a trusted file-transfer location available to the TV.
- 2 Install a reputable file manager from the TV's app store.
- 3 In Android settings, allow **Install unknown apps** for that file manager.
- 4 Open the APK in the file manager and confirm installation.
- 5 Turn off the file manager's unknown-app permission afterward.

Some TV vendors disable USB APK installation. Use ADB when the package installer is unavailable.

Update, replace, or remove Plezy

Install a newer APK signed by the same key with `adb install -r`. If ADB reports `INSTALL_FAILED_UPDATE_INCOMPATIBLE`, the installed copy uses a different signing certificate. Back up anything needed, then replace it:

```
adb uninstall com.edde746.plezy
adb install ".\dist\android-tv\plezy-android-tv-arm64-v8a-official-2.10.0.apk"
```

To remove Plezy without reinstalling, run only the `adb uninstall` command. Uninstalling clears Plezy's local settings and downloaded data.

Model family 2: Amazon Fire TV / Fire TV Stick (Fire OS)

Compatibility and package choice

The Fire TV APK supports Android-based Fire OS 6 or newer (Android API 25+). This includes many current Fire TV sticks, cubes, and television sets, but not Fire OS 5 devices such as the second-generation 2016 Fire TV Stick. The universal APK contains both `armeabi-v7a` and `arm64-v8a` native libraries.

Check the device model and OS in **Settings > My Fire TV > About** before using these instructions.

Install-file status: YES. `dist/fire-tv/plezy-fire-tv-universal-2.10.0-release.apk` is already made and ready for private sideloading on compatible Fire OS devices. It is locally sideload-signed, not signed for Amazon Appstore publication.

Vega OS models require the separate Vega package

This APK cannot be installed on a Vega OS device. Amazon introduced Vega OS with the Fire TV Stick 4K Select and uses it on newer models such as the 2026 Fire TV Stick HD. Vega applications are separate React Native packages (`.vpkg`), not Android APKs. If the About screen reports Vega OS, skip the ADB/APK steps below and use [Model family 3](#).

Build the Fire OS APK

Prerequisites are the same as the Android TV build: Flutter 3.44+, Java 21, Android Platform 36, Build Tools 36.1.0, NDK 29.0.14206865, CMake 4.1.2, and Platform Tools. Build from the repository root:

```
.\scripts\build_fire_tv.ps1 -AndroidSdk "C:\Android\Sdk"
```

If you installed the workspace-local toolchain, use:

```
.\scripts\build_fire_tv.ps1 -AndroidSdk ".\android-sdk"
```

The result is `dist/fire-tv/plezy-fire-tv-universal-2.10.0-release.apk`. The script compiles the dedicated Amazon variant and checks its version code, minimum/target SDK, Leanback launcher, Fire TV ARM ABIs, signature, and SHA-256.

The local release is signed with the Android debug certificate for private sideloading unless `android/key.properties` provides your publisher keystore. It is not signed for Amazon Appstore submission.

Enable Fire OS developer options

- 1 Put the Fire TV and development computer on the same trusted local network.
- 2 Open **Settings > My Fire TV > About**.
- 3 Highlight the device name and press the remote's **Select/OK** button seven times. A message confirms that developer options are enabled. Older Fire OS versions may already display the menu.
- 4 Go back to **My Fire TV > Developer Options**.
- 5 Turn on **ADB Debugging**.
- 6 Turn on **Apps from Unknown Sources**, or grant **Install Unknown Apps** if that is how the firmware labels it.
- 7 Find the Fire TV IP address under **Settings > My Fire TV > About > Network**.

Connect and install the APK with ADB

From the repository root, replace FIRE_TV_IP with the address shown by the device:

```
.\.android-sdk\platform-tools\adb.exe connect FIRE_TV_IP:5555
.\.android-sdk\platform-tools\adb.exe devices
.\.android-sdk\platform-tools\adb.exe install -r
"..\dist\fire-tv\plezy-fire-tv-universal-2.10.0-release.apk"
```

Accept the debugging authorization dialog on the television and optionally select **Always allow from this computer** only when it is your trusted computer. After **Success**, launch Plezy from **Apps > My Apps/My Library, Recents**, or **Settings > Applications > Manage Installed Applications > Plezy > Launch application**.

If ADB reports **INSTALL_FAILED_UPDATE_INCOMPATIBLE**, Plezy is already installed with a different signing certificate. Uninstalling it clears local settings and downloads:

```
.\.android-sdk\platform-tools\adb.exe uninstall com.edde746.plezy
.\.android-sdk\platform-tools\adb.exe install "..\dist\fire-tv\plezy-fire-tv-universal-2.10.0-release.apk"
```

Update, replace, or remove Plezy

Updates built with the same signing key can be installed with the existing `adb install -r` command without clearing Plezy data. If the signing key differs, use the uninstall/install commands above. To remove Plezy without reinstalling, run only:

```
.\.android-sdk\platform-tools\adb.exe uninstall com.edde746.plezy
```

For security, turn off **ADB Debugging** and unknown-source installation after installation or maintenance is complete.

Model family 3: Amazon Fire TV / Fire TV Stick (Vega OS)

Compatibility and package choice

Use this section only when **Settings > My Fire TV > About** reports **Vega OS**. Do not use an APK or ADB. Vega OS uses Amazon's React Native for Vega runtime and installs `.vpkg` packages with the Vega CLI.

The prepared target is under `vega-os/`. Its host application loads a package-local TV UI through Amazon's WebView component; no hosted Plezy website is required after installation. The client includes Plex PIN linking, Jellyfin login, home/library/search/detail browsing, series and season navigation, resume, HLS-first playback, direct-play fallback, progress reporting, D-pad focus, media controls, seek, and Back handling.

Install-file status: NO. The installable `.vpkg` has not been compiled because Amazon's Vega SDK does not support this Windows/WSL host. The validated source ZIP and one-command build/install scripts are already made; run them on native macOS or Ubuntu to produce `dist/vega-os/plezy-vega-os-2.10.1-release.vpkg`.

Understand what is already built and what is not

The portable source archive `dist/vega-os/plezy-vega-os-source-2.10.1.zip` is ready. It includes the Vega wrapper, build/install automation, shared TV client, license, and this guide. It is not an installable VPKG.

The current computer is Windows, and Amazon explicitly does not support or test the Vega SDK on Windows or Windows Subsystem for Linux. Therefore the final VPKG must be compiled on a **native macOS 10.15+ or Ubuntu 20.04+** computer. This is a vendor toolchain requirement, not an unfinished source-code step. Amazon also requires final testing on a physical Vega Fire TV before Appstore submission.

Prepare a supported Vega build computer

You need:

- 1 A native Mac or Ubuntu computer with at least 20 GB of free disk space. Do not use Windows or WSL.
- 2 Node.js 18 or newer, Visual Studio Code, and an Amazon Developer account.
- 3 Amazon's current [Vega Developer Tools](#), including the Vega SDK, Vega CLI, Vega Studio extension, and VDA device utility.
- 4 The repository or the extracted Vega source ZIP.

Amazon's command-line installer is:

```
curl -fsSL https://sdk-installer.vega.labcollab.net/get_vvm.sh | bash
source ~/vega/env
```

Restart VS Code and the terminal after installation. Then verify and update the toolchain:

```
vega --version
vega update
vega sdk config doctor
vega project list-templates
```

The template list must contain `hello-world`. Vega SDK 0.23 currently uses React Native 0.72 for that template. Because Vega is an open-beta SDK, use the latest matching SDK/device software and re-run validation whenever Amazon updates it.

Validate the source on Windows before handoff

These commands do not compile a VPKG, but they verify the wrapper, package identity, local WebView entry point, exit bridge, manifest-service preparation, release build mode, dependency pin, and Fire TV architecture:

```
.\scripts\validate_vega_os.ps1
.\scripts\package_vega_os_source.ps1
```

The second command refreshes the source ZIP and its SHA-256 file under `dist/vega-os/`.

Build the release VPKG on macOS or Ubuntu

From the extracted repository/source root:

```
chmod +x vega-os/build.sh vega-os/install.sh
PLEZY_VEGA_BUILD_NUMBER=2 ./vega-os/build.sh
```

The build script:

- 1 Generates Amazon's official `hello-world` project as `vega-os/generated/PlezyVega` with package ID `com.edde746.plezy.vega`.
- 2 Bundles the Plex/Jellyfin TV client at `assets/index.html`.
- 3 Installs `@amazon-devices/webview@~3.3.0`.
- 4 Adds Amazon's documented input, media, media-metrics, buffer, transform, audio, accessibility, and Group-IPC services to `manifest.toml`.
- 5 Validates the manifest with VPT.
- 6 Builds a Release VPKG for the physical Fire TV `armv7` architecture with explicit version and build metadata.
- 7 validates the resulting VPKG metadata, copies it to `dist/vega-os/plezy-vega-os-2.10.1-release.vpkg`, and writes `SHA256SUMS.txt`.

For every update, increase both values; Amazon rejects Appstore downgrades:

```
PLEZY_VEGA_VERSION=2.10.2 PLEZY_VEGA_BUILD_NUMBER=3 ./vega-os/build.sh
```

You can also open `vega-os/generated/PlezyVega` in VS Code after the first scripted preparation, select the Vega Studio icon, choose **Release**, select the device, and run **Vega Project: Build and run Vega project**.

Register and enable Developer Mode on the Vega Fire TV

Vega Developer Mode is different from Fire OS ADB mode. The device must be registered to your Amazon Developer account before sideloading.

- 1 Connect the Fire TV to a display and complete its normal registration and all over-the-air updates.
- 2 Connect the Vega device to the build computer over USB. A Fire TV Stick can draw up to 1 A at 5 V during development; if it reboots, use a powered USB 3 hub or a USB Y-cable with the Amazon power adapter.
- 3 Open **Settings > My Fire TV > About** and record the **Serial Number**. This is the Device Serial Number (DSN), not the IP address.
- 4 While still in **About**, highlight the device name and press the remote's center/Select button seven times. Press Back; **Developer Options** now appears under **My Fire TV**.

- 5 Open **Developer Options** and use its QR code/link to sign in to the Amazon Vega Developer Portal. Add the DSN under registered devices.
- 6 Return to **Developer Options > Developer Mode** and enable it.
- 7 Allow the device to reboot. Return to **Developer Options** and confirm that Developer Mode says **Enabled**.
- 8 On the build computer, verify that VDA sees the device:

```
vega device list
vega device info --device DEVICE_SERIAL_NUMBER
vega device is-connected --device DEVICE_SERIAL_NUMBER
```

If more than one device is attached, keep the DSN for the install command. Developer Mode can be disabled only by a factory reset on current Vega devices; do not enable it on a device you cannot reset later.

Install and launch Plezy

With one Vega device connected:

```
./vega-os/install.sh dist/vega-os/plezy-vega-os-2.10.1-release.vpkg
```

With multiple connected devices:

```
./vega-os/install.sh dist/vega-os/plezy-vega-os-2.10.1-release.vpkg DEVICE_SERIAL_NUMBER
```

The script uses VPT to display package metadata, installs the VPKG, and launches component `com.edde746.plezy.vega.main`. The equivalent raw commands are:

```
vega exec vpt info dist/vega-os/plezy-vega-os-2.10.1-release.vpkg --json
vega device -d DEVICE_SERIAL_NUMBER install-app --packagePath
dist/vega-os/plezy-vega-os-2.10.1-release.vpkg
vega device -d DEVICE_SERIAL_NUMBER launch-app --appName com.edde746.plezy.vega.main
```

The package deliberately uses a non-Amazon ID. Developer Mode rejects sideload packages whose ID begins with `com.amazon`.

First launch and playback test

- 1 Open Plezy from the Fire TV app list if it is not already running.
- 2 For Plex, choose **Connect Plex**, visit the displayed Plex link on a phone/computer, enter the PIN, select the server, and wait for the home screen.
- 3 For Jellyfin, choose **Connect Jellyfin**, enter the complete server URL (including `https://` or `http://` and any port), username, and password.
- 4 Test D-pad navigation, Select, Back, search keyboard entry, a movie, a resumed item, an episode, Play/Pause, and 30-second left/right seeking.
- 5 Stop playback and confirm that progress appears on the server.
- 6 Test the actual codec, resolution, HDR, audio, and subtitle combinations used in the household. Vega WebView hardware support includes H.264, H.265, VP8, and VP9, but successful direct play still depends on the Fire TV model, display, audio path, server, and media profile. The client requests HLS transcoding first for compatibility and falls back to direct play.

Plezy permits local HTTP requests so it can reach common home Plex/Jellyfin servers. Prefer HTTPS with a valid certificate, keep plain HTTP on a trusted LAN, and never expose an unauthenticated server to the public internet.

Update, remove, troubleshoot, and publish

To update, rebuild with higher version/build values and run the same install command. To remove the sideloaded package:

```
vega device -d DEVICE_SERIAL_NUMBER uninstall-app --appName com.edde746.plezy.vega.main
```

For logs:

```
vega device -d DEVICE_SERIAL_NUMBER shell  
loggingctl log --follow --vpkg com.edde746.plezy.vega
```

If installation fails, run `vega exec vpt validate` on the generated `manifest.toml`, `vega exec vpt info` on the VPKG, confirm the device is registered and Developer Mode is enabled, and ensure the SDK version matches the device software. If the Stick reboots during builds or playback, fix USB power before investigating the app.

For Amazon Appstore publication, upload the release VPKG in the existing app listing's **Amazon Vega TV** section. APK version codes and VPKG build numbers are tracked independently, but each new value must increase within its own package type. Complete physical-device testing and provide the GPL-3.0 corresponding source. More implementation and automation detail is in `vega-os/README.md` (`vega-os/README.md`).

Model family 4: Samsung Tizen TV

Compatibility and package choice

Use this section only for Samsung televisions whose device information reports Tizen TV 6.0 or newer. Samsung televisions do not accept Android APKs. Each physical TV must authorize the developer computer, and the installable `.wgt` must be signed using a Samsung certificate profile that explicitly includes that TV's DUID.

Install-file status: NO. The installable WGT is intentionally not prebuilt because it must be certified for the specific television. Samsung developer-mode installation requires a Samsung certificate profile whose distributor certificate explicitly authorizes the target TV by DUID. The complete Plezy Tizen 2.10.3 source ZIP is already made, but you must connect the intended TV, include its DUID when creating the Samsung certificate profile, and build/sign the WGT for that TV.

Understand Samsung signing

Samsung TVs accept only signed WGT packages. The signature must use a Samsung certificate profile authorized for the target TV. That profile's distributor certificate contains an allow-list of TV device IDs (DUIDs); if the intended TV's DUID is absent, that TV rejects the package. A WGT certified for one TV or one DUID list is therefore not a safe universal installer for other TVs. The repository includes the complete Tizen source plus a validation script; the Tizen extensions for VS Code connect to the intended TV, create a Samsung certificate profile containing its DUID, sign the WGT, and install it.

USB installation of developer widgets is not supported on current Samsung TVs. Use the Tizen extensions for VS Code over the local network.

Install the current VS Code extensions

Samsung now recommends its VS Code extension as the primary Tizen development environment. The new extension automatically manages its required SDK components; the standalone Tizen Studio IDE is not required for this workflow.

- 1 Install the current 64-bit [Visual Studio Code](#).
- 2 In VS Code, open **Extensions** with `Ctrl+Shift+X`.
- 3 Install the official [Tizen Extension](#). It supports Tizen Web projects and automatically manages the core SDK.
- 4 Install Samsung's [Tizen TV extension](#). It adds Samsung TV packaging, certificate, launch, and debugging commands.
- 5 Reload VS Code when prompted.

You can install both from PowerShell if the `code` command is available:

```
code --install-extension tizen.vscode-tizen-csharp
code --install-extension tizensdk.tizentv
```

After installation, a Tizen icon appears in the VS Code Activity Bar. Allow the extension to download or update required SDK components when prompted.

Enable Developer Mode on the TV

- 1 Connect the TV and development computer to the same network.
- 2 Open the TV's **Apps** panel and then **App Settings**.
- 3 Enter 12345 with the remote/on-screen number pad to open Developer Mode.

- 4 Turn Developer Mode **On**.
- 5 Enter the development computer's LAN IPv4 address.
- 6 Reboot the TV fully.
- 7 Confirm that **Develop Mode** appears in the Apps panel.

Samsung changes menu placement between model years. If 12345 does not open the dialog, consult the developer-mode instructions for the exact model/firmware.

Connect VS Code to the TV

- 1 In VS Code, select the **Tizen** icon in the Activity Bar.
- 2 Under **Actions**, select **Connect Device**.
- 3 In **Remote Devices**, select **+ Create Custom Connection**.
- 4 Enter a recognizable device name, the TV's IP address, and port 26101.
- 5 Add the device, then select **Connect**.
- 6 Accept any authorization prompt shown by the TV or extension.
- 7 Confirm the TV appears as the selected connected target in the Tizen sidebar.

Port 26101 is an internal Tizen device port. Do not create a router port-forwarding rule; the TV and computer should communicate only on the same trusted LAN.

Create the Samsung certificate profile

Connect the TV before creating the certificate so its DUID is available:

- 1 In the Tizen sidebar under **Actions**, select **Create Certificate**. If that action is not shown, open the Command Palette with **Ctrl+Shift+P** and run **Tizen TV: Run Certificate Manager**.
- 2 Enter a profile name that you will recognize later.
- 3 Select **Create Samsung Certificate**, not a generic Tizen certificate.
- 4 Enter the requested author details and choose the appropriate TV distributor options.
- 5 Select the checkbox for the connected TV's DUID.
- 6 Select **Create** and sign in to your Samsung Developer account when prompted.
- 7 Confirm that the new profile is active.
- 8 Back up the author certificate and its password. Losing either one prevents signature-compatible updates to an installed copy of Plezy.

Validate the Plezy source

From the repository root, run the local validation script before packaging:

```
.\scripts\build_samsung_tizen.ps1
```

Without a certificate or standalone CLI, this still performs JavaScript syntax, package, and API checks and creates `dist/samsung-tizen/plezy-samsung-tizen-source.zip`.

The current Samsung source is version **2.10.3** (2.10.3-samsung.4). It adds direct playback, automatic season/episode loading, Play Next, faster D-pad focus, and bounded poster rendering. Rebuild any WGT reporting 2.10.1 or 2.10.2 before testing this update.

Build the signed WGT in VS Code

- 1 In VS Code, select **File > Open Folder** and open the repository's `samsung-tizen` folder. Its `config.xml` must be at the workspace root.
- 2 Confirm that your Samsung certificate profile is active and the TV remains connected.
- 3 Open the Command Palette with `Ctrl+Shift+P`.
- 4 Run **Tizen TV: Build Signed Package**.
- 5 Wait for the Tizen output channel to report a successful build.
- 6 Locate the generated `.wgt` in the opened workspace. For consistency with this repository, copy it to `dist/samsung-tizen/plezy-samsung-tizen.wgt`.

The application ID is `p1ezytv001.PlezyTV`. Do not change the application ID between updates unless you intentionally want the TV to treat the build as a separate application.

Install and launch from VS Code

- 1 Keep the `samsung-tizen` folder open and the TV selected as the connected target.
- 2 In the Tizen sidebar under **Actions**, select **Run Project**. You can alternatively run **Tizen TV: Launch Application** from the Command Palette.
- 3 The extension builds if necessary, signs with the active Samsung profile, installs the WGT, and launches Plezy on the selected TV.
- 4 Check the Tizen output channel for the install and launch result.

The previous version is normally replaced when a newer package with the same application ID and compatible author certificate is installed.

Troubleshoot installation

If installation fails with error 1010, check all of the following:

- The TV is still in Developer Mode and was rebooted after enabling it.
- The computer IP saved on the TV is correct.
- The Tizen sidebar shows the intended TV as connected and selected.
- The WGT was signed with a **Samsung** certificate profile, not a generic Tizen profile.
- The certificate profile contains the target TV's DUID.
- The author certificate matches the certificate used for the already-installed version.
- The TV firmware, VS Code, Tizen Extension, and Tizen TV extension are current.

If installation reports **1010**, certificate or DUID configuration is the most common cause. If it reports **1013**, retry launching and reboot the TV if necessary. If the device cannot connect, confirm both devices are on the same LAN, Developer Mode contains the correct computer IPv4 address, and no host firewall is blocking the extension.

First launch and sign-in

- **Plex:** Select **Connect Plex**, open `https://plex.tv/link` on another device, and enter the four-character code. If the account has multiple servers, choose one on the TV.
- **Jellyfin:** Enter the full server URL, username, and password. Prefer a valid trusted HTTPS certificate. A self-signed HTTPS certificate is commonly rejected by the TV web runtime; use a trusted certificate or LAN HTTP if appropriate for your network.

The Tizen client requests a Samsung-profiled HLS transcode (H.264 video with compatible television audio) first and falls back to direct play if transcoding fails. Actual codec, HDR, audio, and subtitle support varies by TV model and by the Plex/Jellyfin server's transcoder.

Troubleshoot the Samsung interface and playback

- **App is small in the upper-left corner:** the TV is still running the older 2.10.1 WGT. Build and sign 2.10.3 with the same author certificate, install it over the existing application, and relaunch. If the TV does not replace it, uninstall the old developer copy and install the newly signed WGT again.
- **Home reports 404:** confirm the installed WGT is 2.10.3. This version tries Plex home hubs and automatically falls back to the server's On Deck and Recently Added endpoints when a server does not expose the primary hub route.
- **Video does not start:** confirm Plex Media Server transcoding is enabled and that its temporary transcode directory is writable. The app first requests TV-compatible HLS and then tries the original file. If both fail, note the AVPlay error shown on the TV and compare its time with the Plex Media Server log.
- **Update appears unchanged:** VS Code can leave an old WGT in a previous [Debug](#) output folder. Re-run **Tizen TV: Build Signed Package**, verify the generated package embeds version 2.10.3, and install that newly generated file.

Access tokens are stored in the application's private Tizen local storage. Signing out removes the saved session. Do not install builds from untrusted sources.

Samsung model and feature limitations

The Samsung port includes the essential living-room flow but is not yet feature-identical to the mature Flutter client.

Included:

- Plex link-code authorization and multi-server selection
- Jellyfin server/password authorization
- Direct playback from Continue Watching, On Deck, and playable Recently Added cards
- Automatic seasons for shows, automatic episodes for seasons, and direct episode selection
- Show-level Play First/Next/Resume Episode selection based on watch progress
- Home shelves, libraries, search, item details, seasons, and episodes
- Optimized D-pad focus navigation, bounded poster rendering, and Samsung remote media keys
- Resume position, seek, play/pause, progress reporting
- Samsung AVPlay HLS playback with direct-play fallback

Not yet included:

- Offline downloads
- Live TV/DVR guide
- Watch Together and Plezy Remote
- External-player launch
- Advanced subtitle/audio selection and Plezy's mpv shaders
- Store publishing metadata and Samsung Seller Office submission

Model family 5: Roku TV / Roku player

Compatibility and package choice

Roku OS uses SceneGraph and BrightScript; it cannot install the Android APK. The developer ZIP produced here is directly sideloadable on a physical Roku in Developer Mode. A public Streaming Store .pkg must be encrypted and signed by cryptographic hardware in a linked Roku device.

Install-file status: YES. `dist/roku/plezy-roku-2.10.0-roku.2.zip` is already made and ready for Roku's Developer Application Installer. It is a developer-sideload ZIP, not a public Streaming Store package.

Validate and build

Install Node.js 20+ and npm, then run from the repository root:

```
cd .\roku
npm ci
npm run check
cd ..
.\scripts\build_roku.ps1
```

The output is `dist/roku/plezy-roku-2.10.0-roku.2.zip`. The build verifies BrightScript syntax, SceneGraph XML, required manifest/image entries, TLS guardrails, and that `manifest` is at the ZIP root.

Enable Developer Mode

- 1 Put the Roku and computer on the same trusted local network.
- 2 From the Roku home screen, press **Home** three times, **Up** twice, then **Right, Left, Right, Left, Right** without long pauses.
- 3 On the Developer Settings screen, write down the displayed device URL/IP.
- 4 Choose **Enable installer and restart**.
- 5 Read and accept Roku's Developer Tools License Agreement.
- 6 Create a case-sensitive developer web-server password and let the Roku reboot.

If the screen does not appear, return to Home and repeat the sequence at a steady pace.

Sideload the ZIP

- 1 Open the displayed Roku URL in a browser on the same LAN, normally `http://ROKU_IP_ADDRESS`.
- 2 Log in with user name `rokudev` and the password created above.
- 3 In **Development Application Installer**, click **Upload**.
- 4 Select `dist/roku/plezy-roku-2.10.0-roku.2.zip`.
- 5 Choose ZIP or SquashFS compression if prompted, then click **Install**.
- 6 Plezy launches and is placed in the bottom row of the Roku home screen.

Only one developer-sideloaded Roku app can be installed at a time. Uploading another development app replaces Plezy; normal Streaming Store apps are unaffected.

First launch and sign-in

- **Plex:** Choose **Connect Plex**, open <https://plex.tv/link> on a phone/computer, enter the TV code, then choose a server if prompted.
- **Jellyfin:** Choose **Connect Jellyfin** and enter the full server URL, username, and password with Roku's voice-enabled keyboard.

Plezy requires a trusted certificate for HTTPS and intentionally does not disable host or peer verification. A self-signed Jellyfin certificate may be rejected; use a publicly trusted certificate, install an appropriate trusted setup at the server, or use LAN HTTP only on a private trusted network.

Update or remove the developer app

Roku permits only one sideloaded development app and may reject an identical re-upload. Increment `build_version` in `roku/manifest`, rebuild, and upload the new ZIP to update Plezy. To remove it, highlight Plezy on Home, press **Options** (*), and choose **Remove app**.

Package for publication

For publication, sideload the ZIP, generate or restore a signing key on a linked Roku, and use the device web installer's **Packager** page. Preserve the developer ID, key password, and rekey package; changing developer ID can clear private registry data. Full Roku build, update, removal, troubleshooting, and packaging details are in `roku/README.md` (`roku/README.md`).

Verify downloaded or copied artifacts

Compare each file to the generated checksum list before installing:

```
Get-FileHash -Algorithm SHA256 ".\dist\android-tv\plezy-android-tv-arm64-v8a-official-2.10.0.apk"
Get-Content ".\dist\android-tv\SHA256SUMS.txt"
Get-FileHash -Algorithm SHA256 ".\dist\fire-tv\plezy-fire-tv-universal-2.10.0-release.apk"
Get-Content ".\dist\fire-tv\SHA256SUMS.txt"
```

Use the equivalent artifact and `SHA256SUMS.txt` files under `dist/samsung-tizen/` for Samsung and `dist/roku/` for Roku. The calculated hash must exactly match the recorded value for that filename.

For the Vega source handoff, compare `dist/vega-os/plezy-vega-os-source-2.10.1.zip` with `SHA256SUMS-source.txt`. After a supported Mac/Ubuntu build, compare the VPKG with `dist/vega-os/SHA256SUMS.txt` before installing it.

License

Plezy and this TV work are GPL-3.0. If you distribute APKs, VPKGs, WGTs, Roku ZIPs, or Roku packages, provide the corresponding source and preserve the license notices. The Vega, Samsung, and Roku source archives include `LICENSE.txt` automatically.